

Nós que fazem (Ativas) ou não (Passivas) o Beaconing.

Me-Bands

App BLE (Bluetooth Low Energy): BLE Scanner, nRF Connect

Para crackear aplicações BLE: usar Android

pacotes da pilha protocolar Bluetooth: hCI e GAP no
wireshark



Planeamento de Redes - 18013

Aula 12 - Bluetooth Low Energy

Cofinanciado por:

CENTRO 2020

PORTUGAL
2020

 **UNÃO EUROPEIA**
Fundo Social Europeu

Sumário

- Bluetooth Low Energy
- Camada física e camada de acesso à rede.
- Generic Access Profile (GAP).
- Generic Attribute Profile (GATT).
- Ferramentas de gestão de rede Bluetooth.

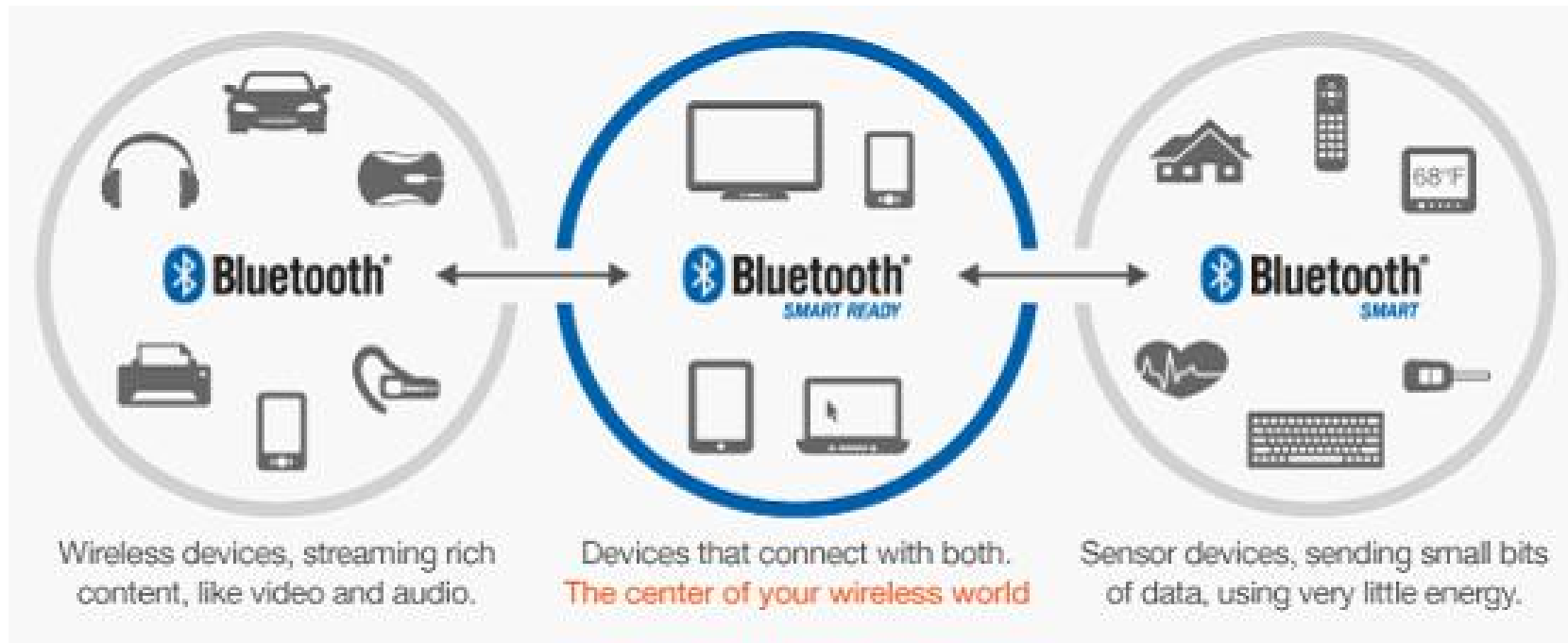


Bluetooth Low Energy

- Norma de redes sem fios desenvolvida pelo Bluetooth Special Interest Group (BT-SIG)
- Subset leve do Bluetooth classico.
- Adicionado ao Bluetooth standard na versao 4.0 of the Bluetooth core specification (adoptado em 2010)
- Concebido para aplicações wireless low-power, low-latency, and low-throughput
- Nao e retro compativel com dispositivos Bluetooth classicos



Single Mode e Dual Mode



BLE vs. Classic BT

Spec.	BLE	BT Classic
Topology	Star	Scatternet
Throughput	0.7 – 2.1 Mbps	0.3 Mbps
Message	8 – 47 bytes	358 bytes
Power	< 15 mA	< 30 mA
Cost	Low	High
Range	50 – 150 m	< 30 m
Latency	3 ms	100 ms



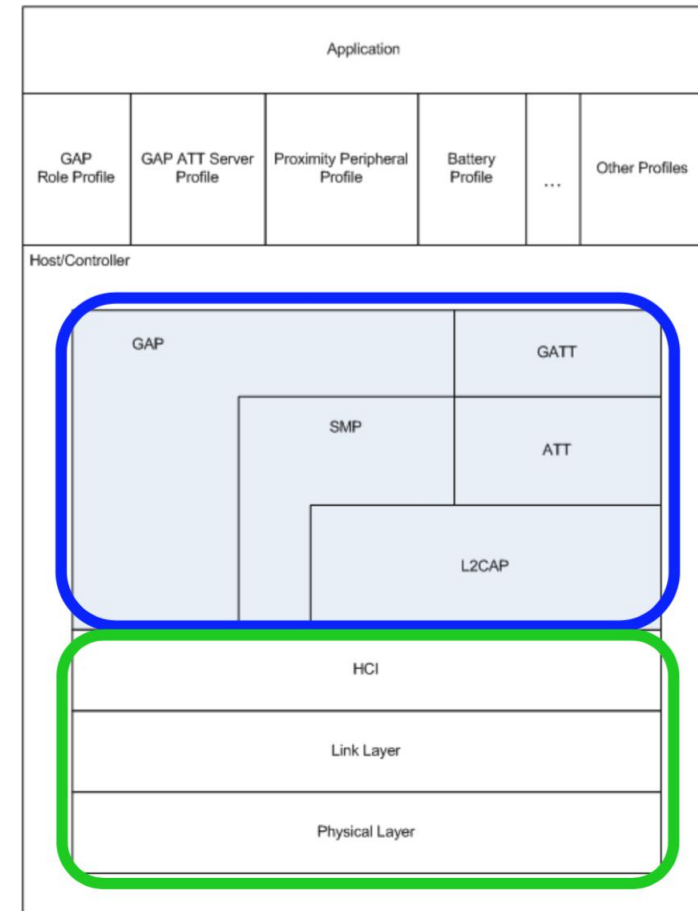
Devices: dois papeis

- Peripheral Device (Server)
 - pequeno
 - Low power
 - Resource constrained
 - Connects to a more powerful (central) device
 - e.g., heart rate monitors, BLE proximity tags
- Central Device (Client)
 - More processing power
 - More memory
 - e.g. Smartphone and tablets



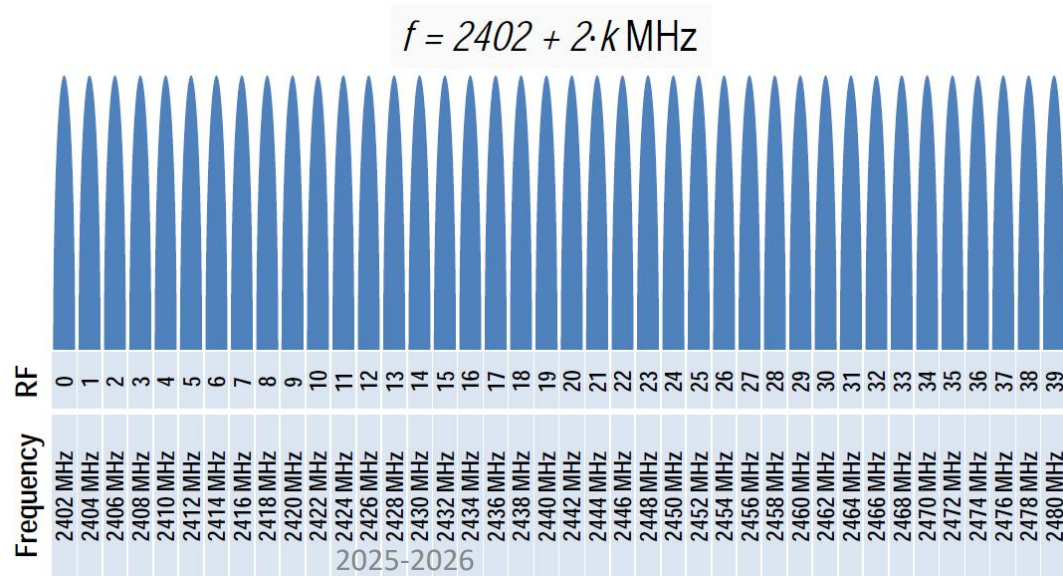
Pilha protocolar BLE

- Two main sections:
 - Controller
 - Host

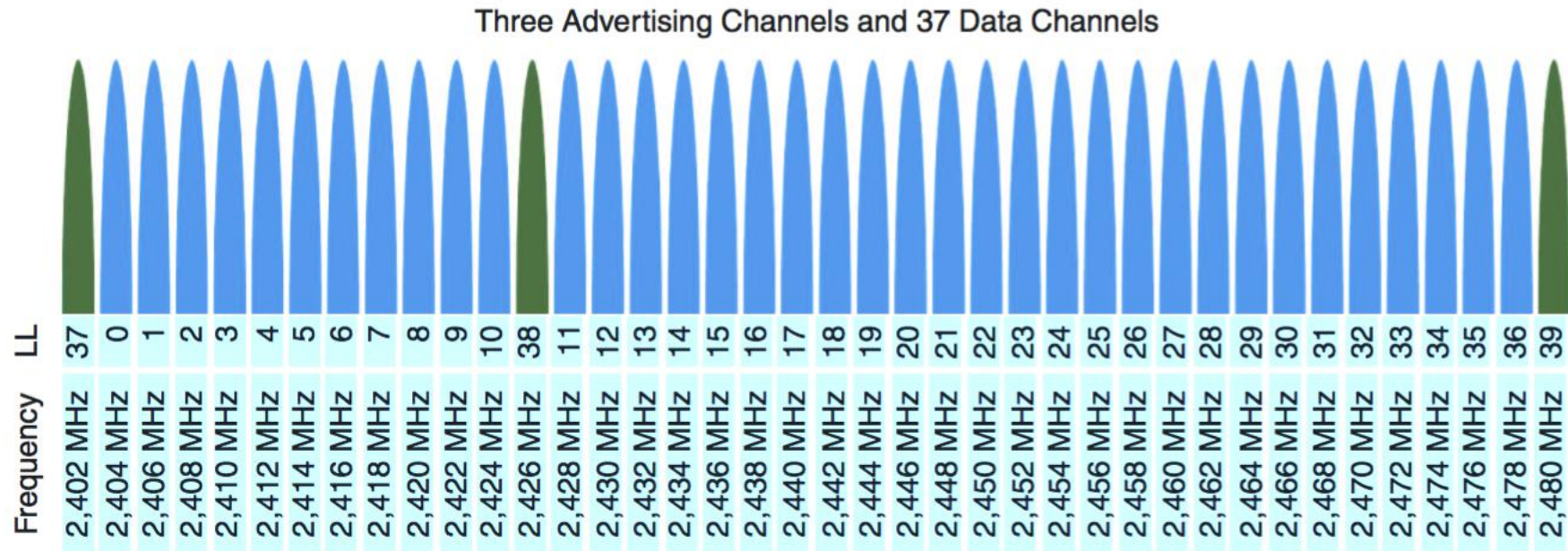


Physical Layer

- 2.4 GHz ISM band
- 1Mbps GFSK
 - Larger modulation index than Bluetooth BR (which means better range)
- 40 Channels on 2 MHz spacing



Physical Layer - Channels

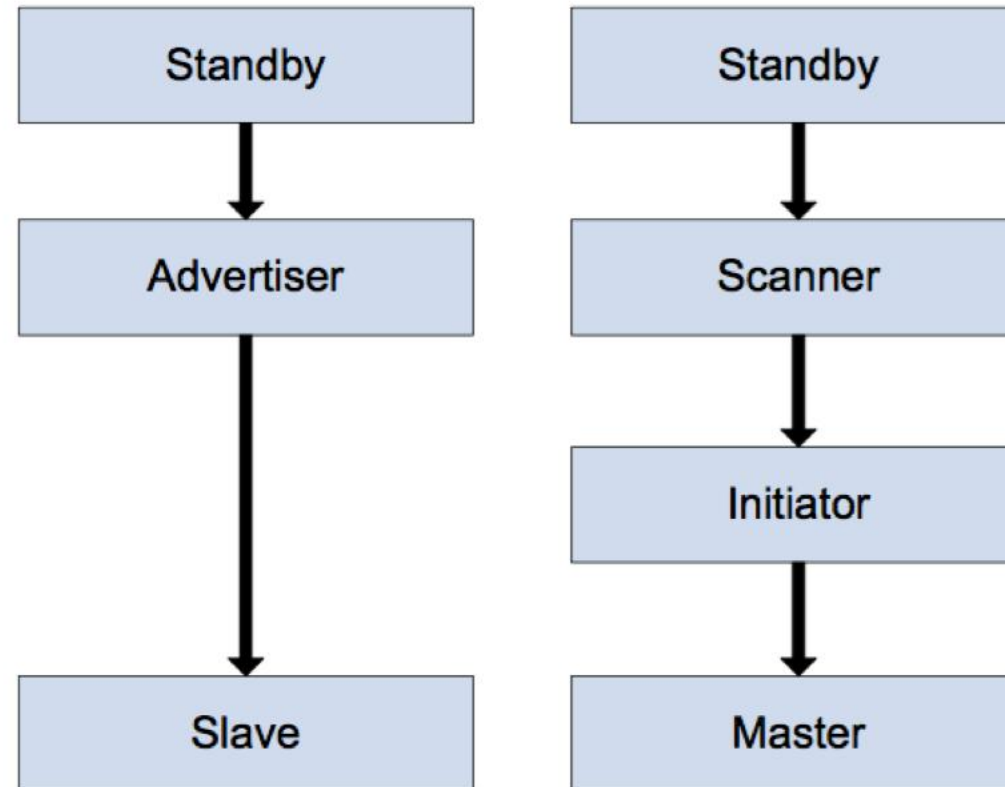


Link Layer - States

- Six possible Link Layer states of a BLE device:
 - Standby: not transmitting or receiving any data, and is not connected to any other device
 - Advertiser: periodically broadcasting advertisements
 - Scanner: actively looking for advertisers
 - Initiator: actively trying to initiate a connection with another device
 - Master: connected to another device as a master
 - Slave: connected to another device as a slave

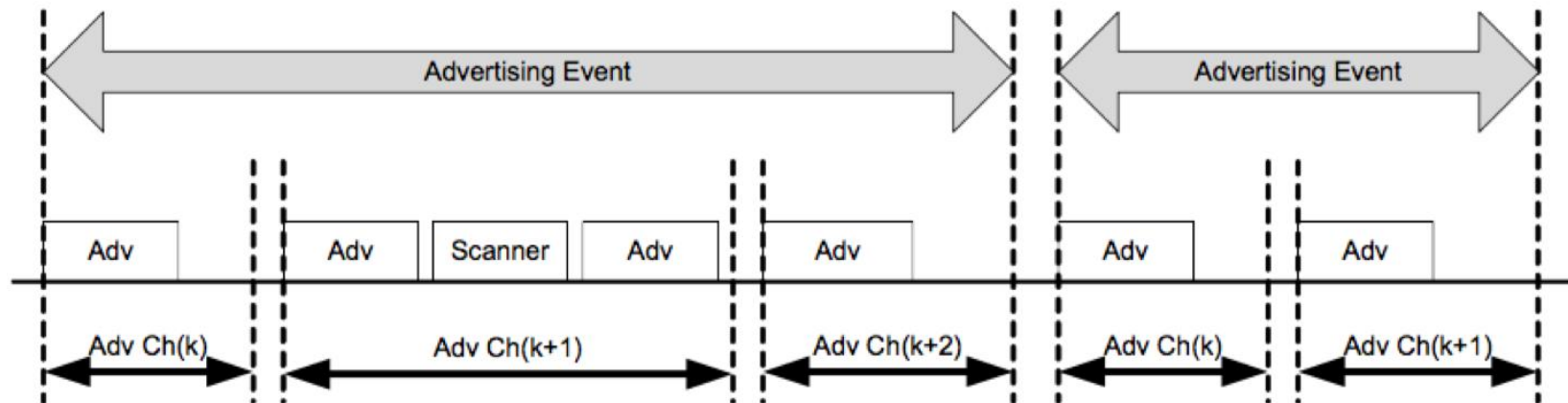


Link Layer - States



Link Layer – Advertisement Events

- During one “advertising event”, an advertisement packet is transmitted on each of the three advertising channels (37,38, and 39)



Link Layer – Advertisement Events

- A device in the advertising state transmits advertising packets
 - Advertising packets can contain a data payload
 - Advertising packets can be directed towards a specific scanner device, or undirected
 - Advertisements can be connectable or non-connectable (and therefore just used for broadcast of data)



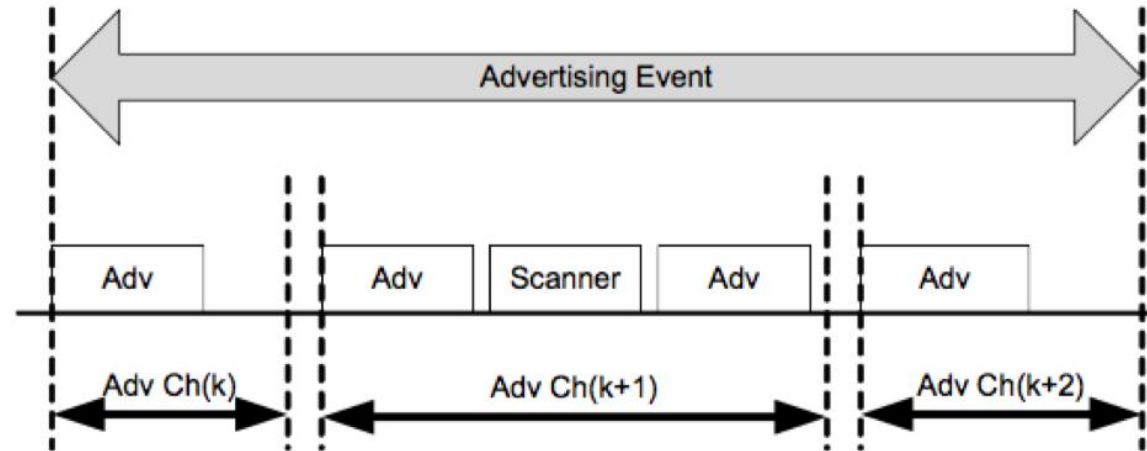
Link Layer – Advertisement Types

- Four types of advertisements:
 - Connectable undirected: any scanner device can initiate a connection with this advertiser
 - Connectable directed: only one specific device can initiate a connection with this advertiser
 - Non-connectable undirected: no devices can initiate a connection with this advertiser; primarily used for general broadcast of data
 - Discoverable undirected: any scanner device can request more information from the advertising device, but no devices can initiate a connection with it



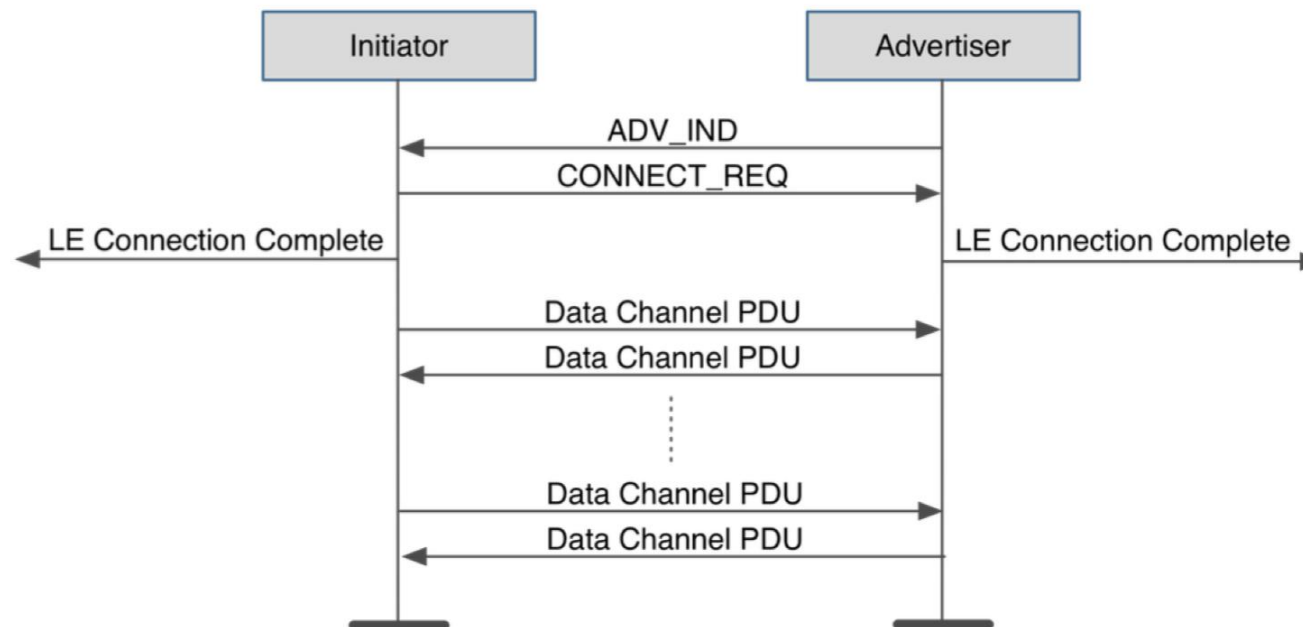
Link Layer - Scanning

- Passive Scanning
 - Scanner listens on advertising channels for advertising packets
 - When an advertisement packet is received, it passes the information up to the host
- Active Scanning



Link Layer – Connection Initiation

- Scanner → Initiator: send a “connection request” packet to the advertiser which contains a set of link layer parameters for the slave device



Link Layer – Connection Parameters

- Channel Map: indicates which data channels are used during the connection
- Hop Increment: random value between 5 and 16 for channel selection algorithm
- Connection Interval: multiple of 1.25ms in range of 7.5ms and 4.0s
- Supervision Timeout: multiple of 10ms in the range of 100ms and 32.0s
- Slave Latency: any value between 0 and 499

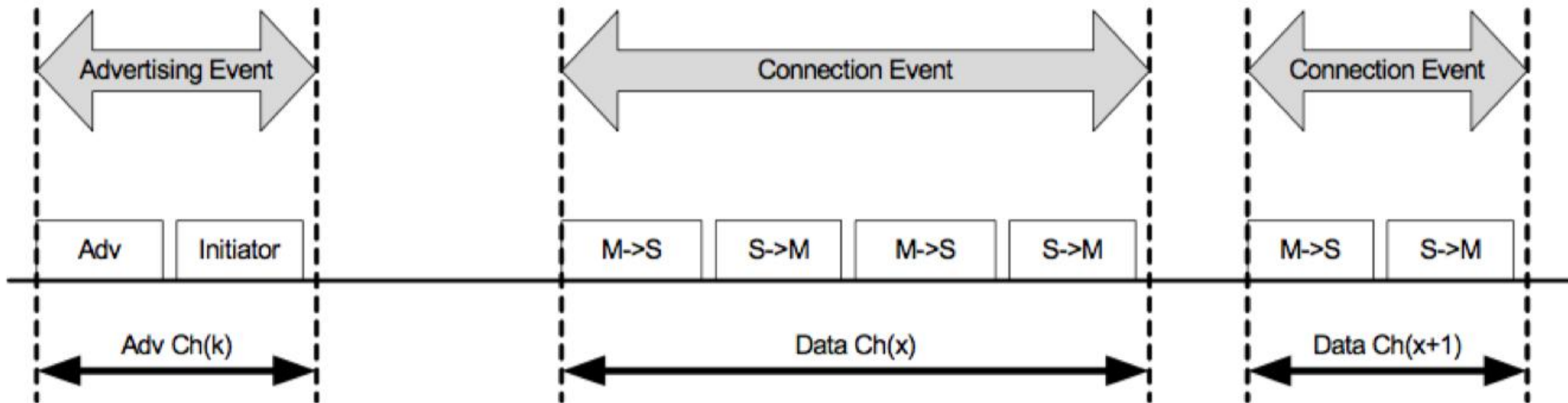


Link Layer – Connection Events

- Connection events occurs periodically, with the connection interval parameter specifying the period
- Each event occurs on one data channel (channels 0-36), with the hop increment parameter determining the next channel for the next event
- Connection events occur even when one (or both) sides have no data to send



Link Layer



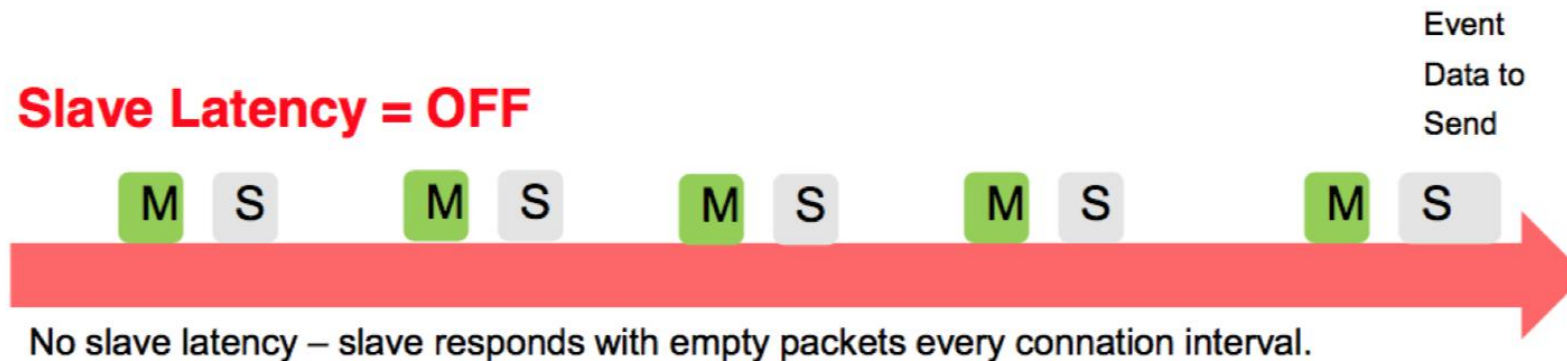
Link Layer – Slave Latency

- Slave latency: the maximum number of connection events that the slave can skip
- Slave latency allows for a slave device to skip connection events if it does not have any data to send
- If slave doesn't respond to master's packet during a connection event, master will resend the packet in subsequent connection events until the slave responds

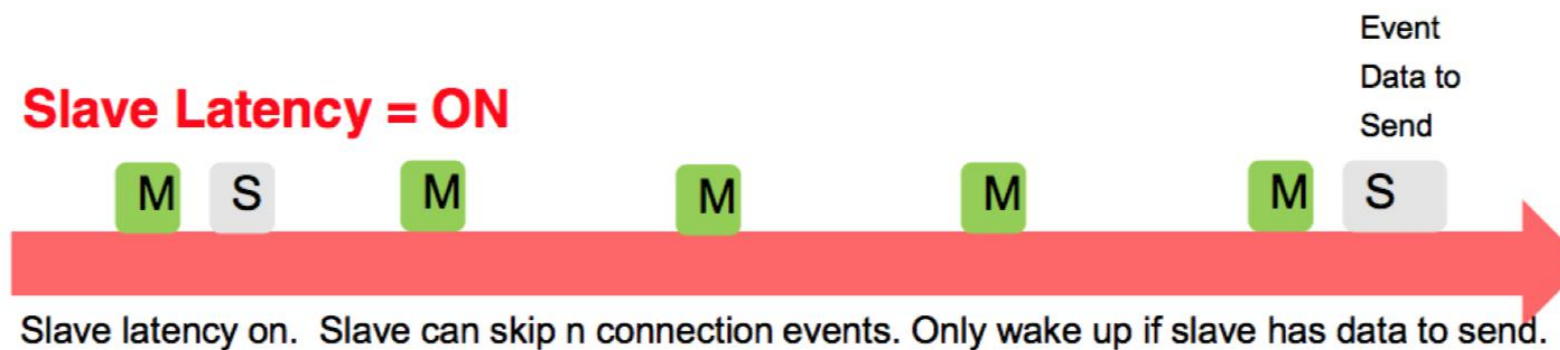


Link Layer

Slave Latency = OFF



Slave Latency = ON



Link Layer – Connection Parameters Tradeoffs

- Short connection interval: higher power consumption, higher throughput, shorter wait for data to be sent
- Long connection interval: lower power consumption, lower throughput, longer wait for data to be sent
- Low / Zero slave latency: higher power consumption, receive data sooner
- High slave latency: lower power consumption, receive data not immediately



Link Layer – Connection Update Request

- If the slave does not like the connection parameters, it can send a connection update request to the master
- Connection update request allows slave device to request a desired connection interval range



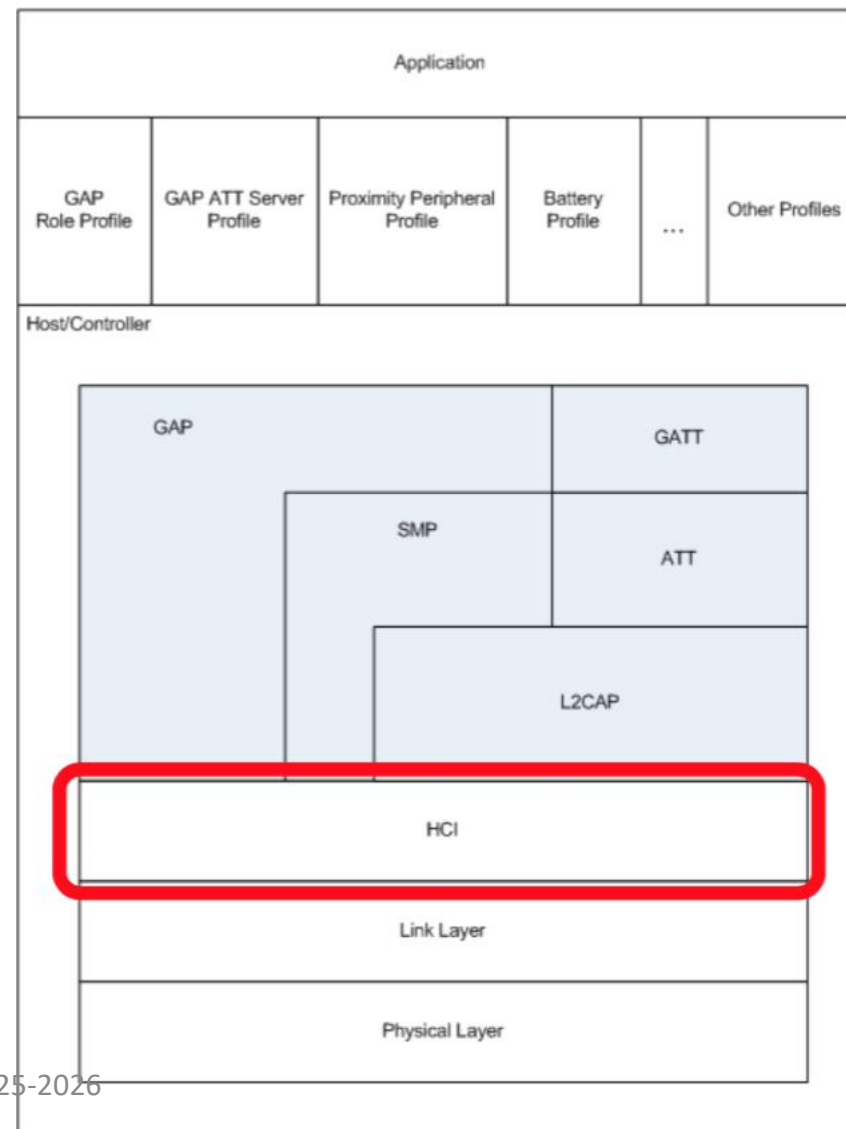
Link Layer – Connection Termination

- A connection can be voluntarily terminated by either the master or the slave for any reason
- Connection can also be terminated as a result of a supervision timeout
 - If supervision timer ever reaches the supervision timeout, the device considers the connection lost, and exits the connection state (returning to the advertising, scanning, or standby state)



Host Controller Interface (HCI)

- HCI doesn't perform any processing
- The host uses the HCI API to communicate with the lower layers



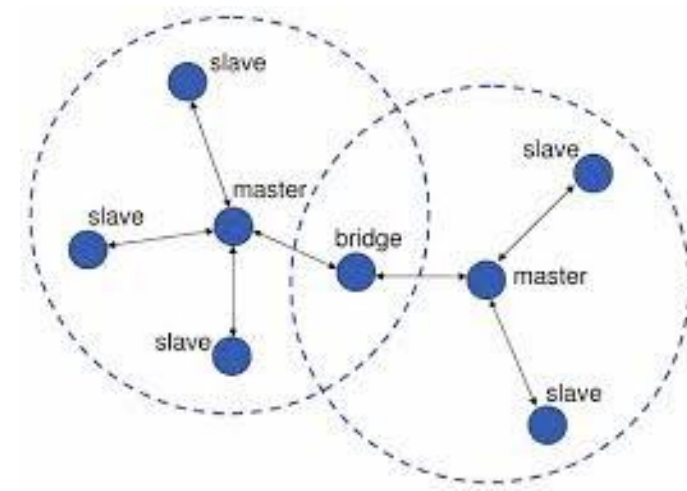
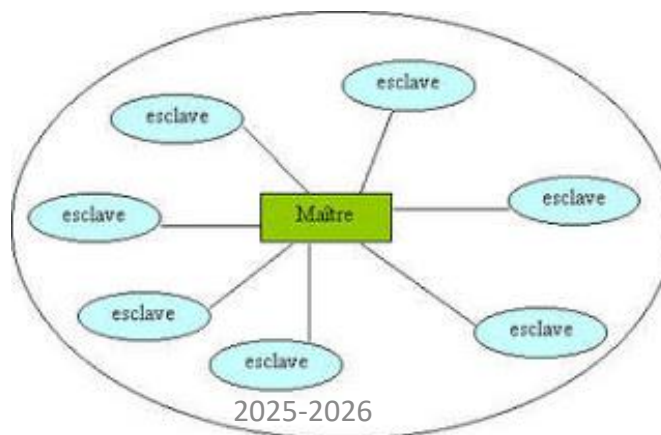
Logical Link Control and Adaptation Protocol (L2CAP)

- Permits upper level protocols and applications to transmit and receive upper layer data packets up to 23bytes in length
- Provides channel management, allowing for logical channels between two endpoints, supported by the link layer



Network Topology

- BLE is a star topology network:
 - Master device “manages” the connection, and can be connected to multiple slaves
 - Slave device can only be connected to one master
 - Each Slave communicates on a separate physical channel with the master
 - Slaves are permitted to have physical links to more than one master at a time
 - A device is permitted to be master and slave at the same time
- Piconet and scatternet



Security Manager Protocol (SMP)

- Performs authentication and key management
- Uses AES-128 as the encryption algorithm for security procedures
- Works with GAP to manage relationships between devices:
 - Pairing: encryption between two devices once a connection has been established between them
 - Authentication: verification that a peer device can be trusted, providing protection against “Man-in-the-Middle” attacks
 - Bonding: long-term relationship between devices; security and identity information is saved for re-use next time the devices are connected



Generic Access Profile (GAP)

- Controls connection and advertising processes.
- Makes a device visible to the world.
- Determines how two devices can/cannot interact.
- Defines the role of a device.



Generic Access Profile (GAP)

- GAP defines a base profile which all Bluetooth devices implement, which ties all the various layers together to form the basic requirements for a Bluetooth device
- GAP also defines generic procedures for connection-related services:
 - Device Discovery
 - Link Establishment
 - Link Management
 - Link Termination
 - Initiation of security features



Generic Access Profile (GAP) - Profile Roles

- The GAP layer works in one of four profile roles:
 - Broadcaster: an advertiser that is non-connectable
 - Observer: scans for advertisements, but cannot initiate connections
 - Peripheral: an advertiser that is connectable and can operate as a slave in a single link layer connection
 - Central: scans for advertisements and initiates connections; operates as a master in a single or multiple link layer connections



Generic Access Profile (GAP)

Temperature Sensor (Broadcaster) → Temperature Display (Observer)



Figure 1 – Temperature Sensor (Broadcaster)



Figure 2 – Temperature Display (Observer)

Watch (Peripheral)

↔ Mobile Phone (Central)



Figure 3 - Watch (Peripheral)



Figure 4 – Mobile Phone (Central)

Generic Access Profile (GAP) – Discoverable Modes

- GAP supports three different discoverable modes:
 - Non-discoverable Mode: No advertisements
 - Limited Discoverable Mode: Device advertises for a limited amount of time before returning to the standby state
 - General Discoverable Mode: Devices advertises continuously
- GAP manages the data that is sent out in advertisement and scan response packets



Generic Access Profile (GAP) – Pairing

- Pairing can be initiated by either the central or peripheral device
- The two devices generate and exchange short-term keys (STK) which can be used to decrypt data packets
- Either device can request to enable “bonding” to create a long-term relationship between the two devices
 - A long-term key (LTK) is generated, exchanged, and stored allowing device to re-encrypt the link quickly upon re-connection, without going through the complete pairing process once again
 - Profile / Service configuration data is remembered, so that the user does not need to re-configure the device every time they re- connect



Generic Access Profile (GAP) – Pairing

- Each device also states its input/output capabilities from among these options:
 - DisplayOnly – no way user can input anything into device, but it can output data
 - DisplayYesNo – user can input “yes” or “no” but nothing else; can also display data
 - KeyboardOnly – user can input a password or PIN, but no display
 - NoInputNoOutput – device has no means for user input, and has no display
 - KeyboardDisplay – device has a means for display as well as for input



Generic Access Profile (GAP) – Pairing

- Based on the combination of the capabilities of the two devices, one of two methods of pairing will be used:
 - Passkey entry – one device will display a randomly generated passkey, while the other will require the user to input the passkey. This allows for an authenticated link (MITM protection)
 - “Just Works” – the pairing process completes without requiring a passkey to be entered. The link will not be authenticated, but is encrypted
- If either one of the two devices does not require authentication, then Just Works will be used by default, allowing the user to skip passkey entry



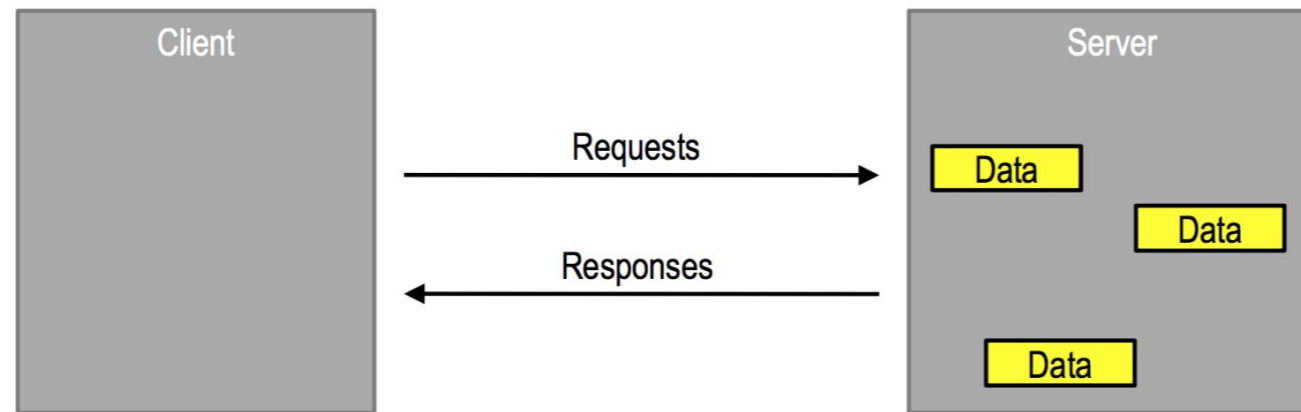
Attribute Protocol (ATT)

- An attribute is a discrete value that has associated with it the following three properties:
 - A handle(address)
 - A type
 - A set of permissions
- ATT defines the over-the-air protocol for reading, writing, and discovering attributes



Attribute Protocol (ATT) – Client/Server Architecture

- Servers have data, Clients want to use this data
- Servers expose data using attributes
- The Client / Server role of a device is independent of the GAP central / peripheral role or the link layer master / slave role



Attribute Protocol (ATT) – Attribute Table Example

- Handle – The address of the attribute in the table
- Type – Tells what the data represents; can be a UUID (universal unique identifier) assigned by the Bluetooth SIG, or a custom type
- Permissions – Enforces if and how the attribute client can access the attribute's value

Handle	Type	Permissions	Value
39	0x2800 (GATT Service UUID)	Read	E0:FF
40	0x2803 (GATT Characteristic UUID)	Read	10:29:00:E1:FF
41	0xFFE1 (Simple Keys state)	(none)	00
42	0x2902 (GATT Client Characteristic Configuration UUID)	Read and Write	00:00

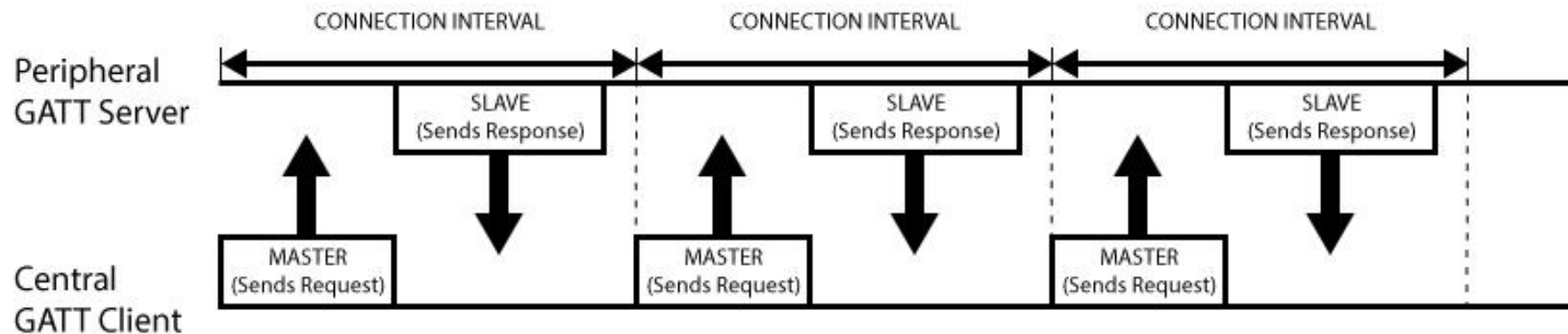
Generic Attribute Profile (GATT)

- Defines how two BLE devices transfer data back and forth using concepts like Services and Characteristics.
- GATT works only after a dedicated connection between two devices has been established.
- Connections are exclusive, i.e. a peripheral can only connect to one central device.
- Communication is two-way.



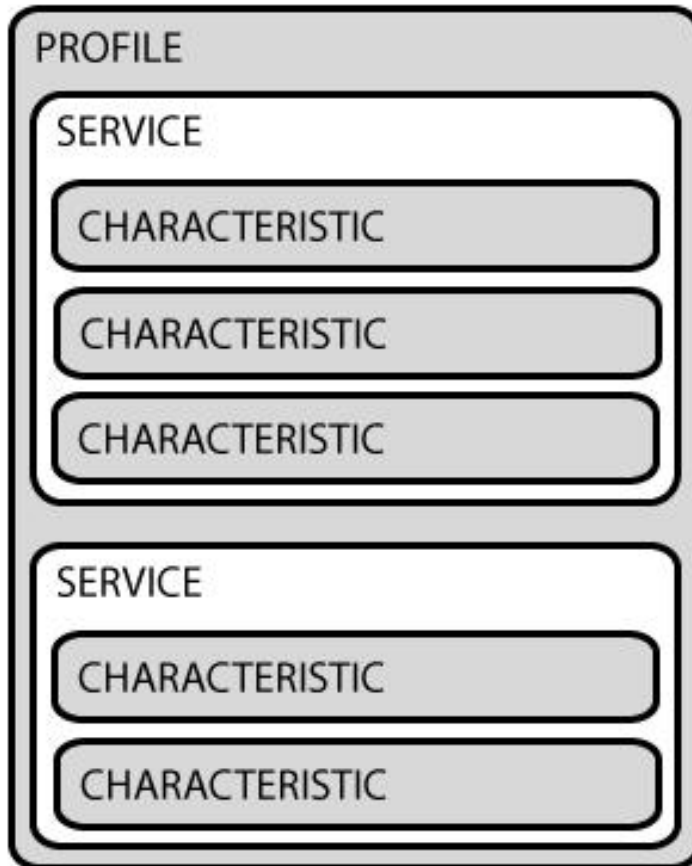
GATT (once they are connected)

- GATT $\approx$ a client-server relationship.
- A peripheral is a GATT Server which holds service and characteristic definitions.
- A central is a GATT Client which sends requests to the server (per



GATT Transaction

- Profiles



is **Profile:**

Predefined collection of Services, compiled by either Bluetooth SIG or a peripheral designer.

Service:

- A collection of characteristics.
- Unique ID called UUID (16 bit – 128 bit)

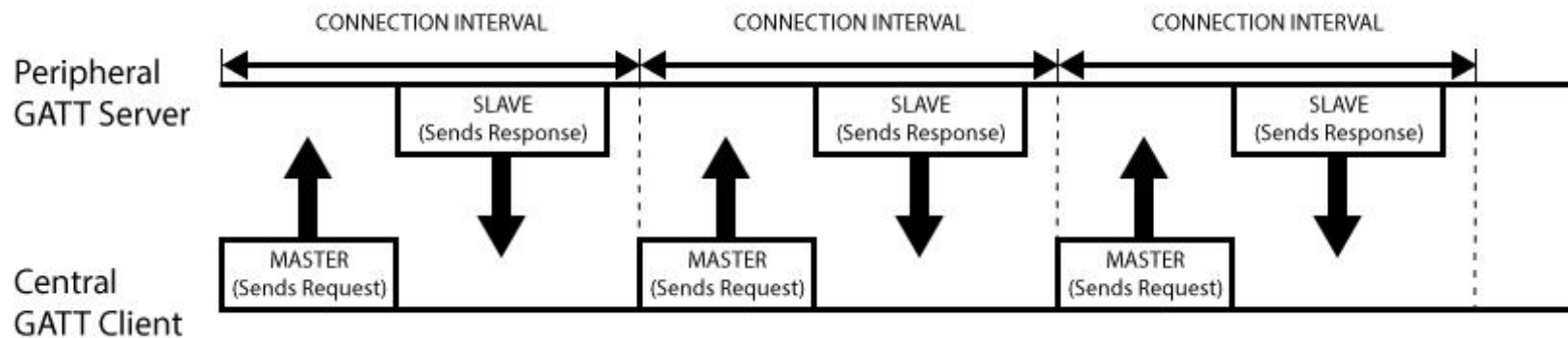
Characteristic:

- Encapsulates a single data point.
- Can be used for sending data back to a peripheral as we can write into a characteristic.



GATT Transaction

- All transactions are started by a GATT Client, which receives a response from the GATT Server.
 - When establishing a connection, a peripheral suggests a connection interval.
 - A central device tries to connect to the peripheral at every connection interval to check for new data.



Connection-oriented, two-way communication

Generic Attribute Protocol (GATT)

- Designed for use by the application or a profile, so that an attribute client can communicate with attribute server
- GATT defines:
 - Procedures for using the attribute protocol (ATT) to discover, read, write, and obtain indications of these attributes
 - The grouping and relationship of characteristics within a service or profile
 - Procedures for configuring the broadcast of attributes

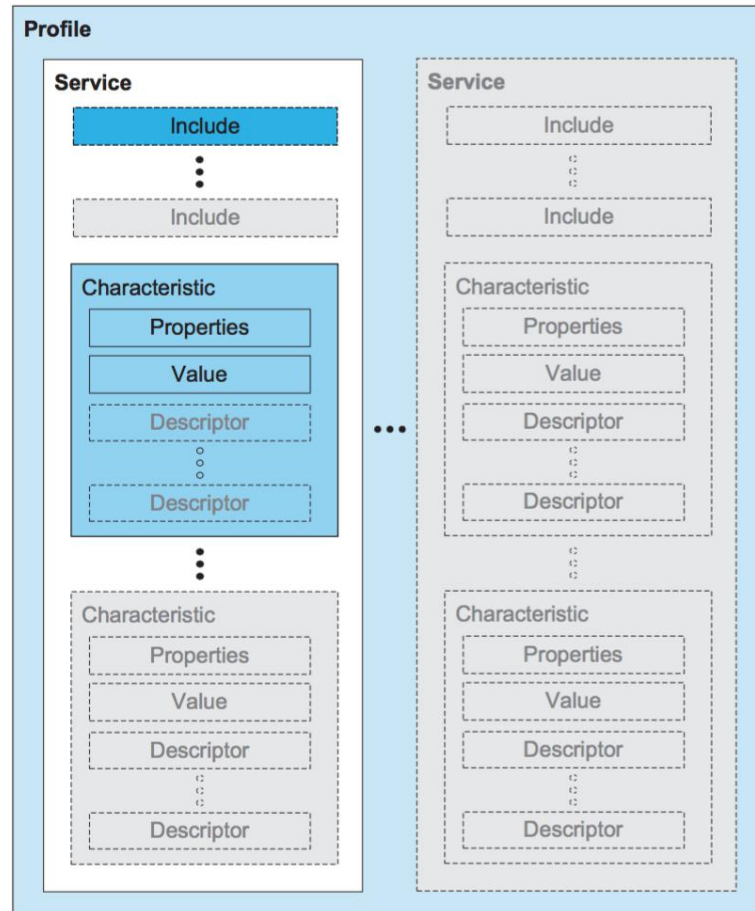


Generic Attribute Protocol (GATT) – Profile Hierarchy

- A profile is composed of one or more “services” necessary to fulfill a use-case
- A service may contain certain attributes called “characteristic values”, which are values used by a service
- A characteristic value must have a mandatory “characteristic declaration” attribute immediately before the value, containing the properties of the characteristic
- Characteristics may also contain optional “descriptor” attributes, with fields such as a configuration or a description



Generic Attribute Protocol (GATT) – Profile Hierarchy



Generic Attribute Protocol (GATT) – Client Commands

- When two BLE devices are in the connected state, the GATT client device can perform several different sub-procedures to communicate with the GATT server device:
 - Discover Characteristic by UUID – search the GATT server for all attributes with type that matches the specified UUID
 - Read Characteristic Value – read the value of the characteristic at the specified handle
 - Write Characteristic Value – write a new value to the characteristic at the specified handle



Applications

- Bluetooth Smart is designed for very low power, at low duty cycle
- It connects to the things we carry around with us:
 - Watches, health and fitness accessories, body sensors
- It allows us to access the things around us:
 - Tags for location, tags as keys, home automation-sensors, home automation-actuators
- It provides communication within a system:
 - Office automation, factory automation, communication within our cars



Applications – Profiles



GATT-Based Specifications					
Profile Specification		Version	Status	Date Adopted	Date Deprecated Date Withdrawn
ANP	Alert Notification Profile	1.0	Active	13 September 2011	
ANS	Alert Notification Service	1.0	Active	13 September 2011	
AIOP	Automation IO Profile	1.0	Active	14 July 2015	
AIOS	Automation IO Service	1.0	Active	14 July 2015	
BAS	Battery Service	1.0	Active	27 December 2011	
BCS	Body Composition Service	1.0	Active	21 October 2014	
BLP	Blood Pressure Profile	1.0	Active	25 October 2011	
BLS	Blood Pressure Service	1.0	Active	25 October 2011	
BMS	Bond Management Service	1.0	Active	21 October 2014	
CGMP	Continuous Glucose Monitoring Profile	1.0	Active	18 November 2014	
CGMP	Continuous Glucose Monitoring Profile	1.0.1	Active	15 December 2015	
CGMS	Continuous Glucose Monitoring Service	1.0	Active	18 November 2014	

Applications – Proximity

- Proximity Profile: defines the behavior when a device moves away from a peer device so that the connection is dropped or the path loss increases above a preset level, causing an immediate alert
- Link Loss Service: uses the Alert Level characteristic to cause an alert in the device when the link is lost.



Applications – Fitness Trackers

- The fitness trackers are only connected to the smartphone when the corresponding smartphone application on the smartphone is running (foreground)
- When the app is running, the tracker actively communicates and synchronizes the activity data (e.g. steps, calories etc.)
- When the app is not running, the connection is terminated, leaving the fitness tracker in advertising state
- Privacy Leakage!



Conclusion

	Classic Bluetooth	Bluetooth Low Energy
range(theoretical max.)	100m	>100m
Over the air data rate	1-3 Mbit/s	1 Mbit/s
Application throughput	0.7-2.1 Mbit/s	0.27 Mbit/s
Active slaves	7	Not defined; implementation dependent
Latency(from a non-connected state)	Typically 100 ms	6 ms
Minimum total time to send data	100 ms	3 ms

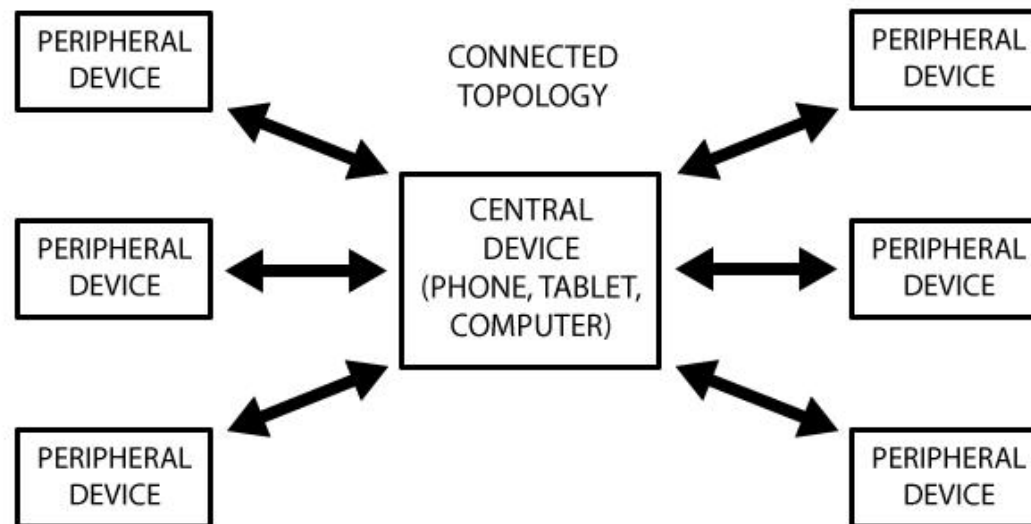
5/30/2026

2025-2026

*

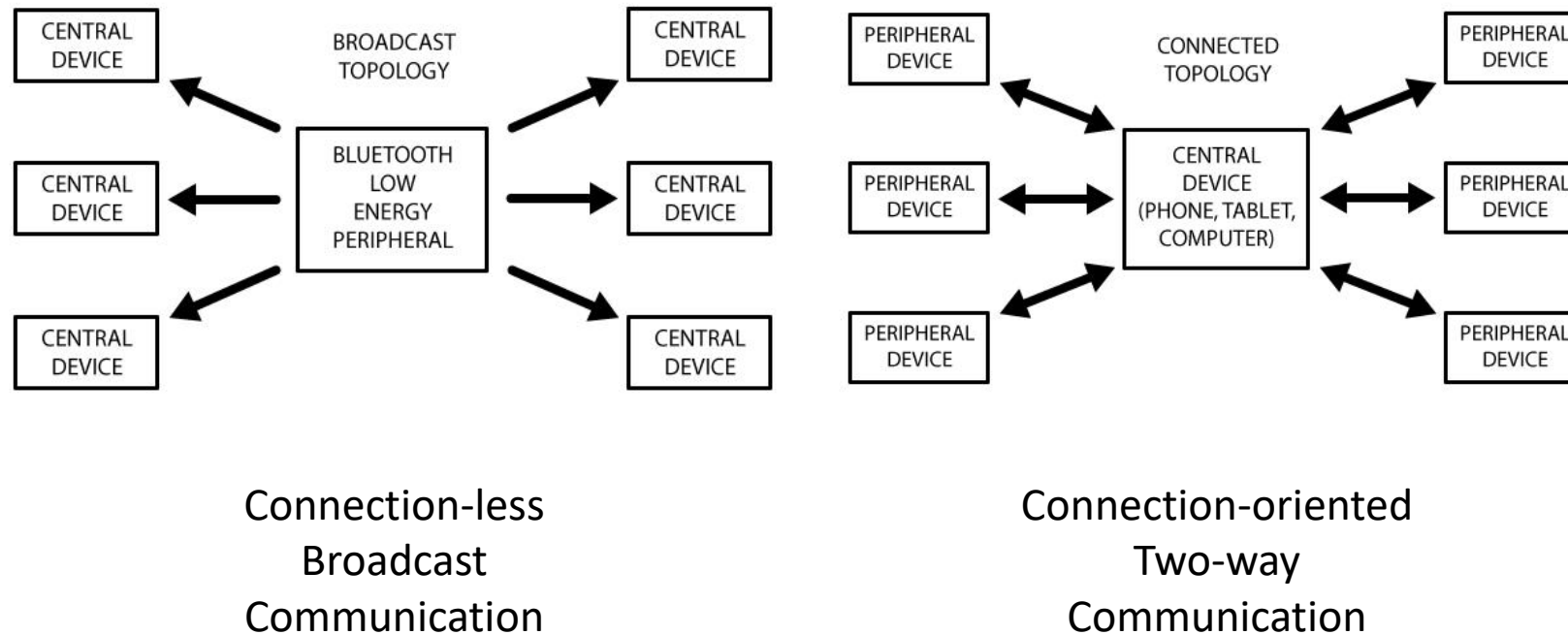
Connected Network Topology

- One central and many peripherals.
- For two peripherals to talk, all messages have to pass through a central.



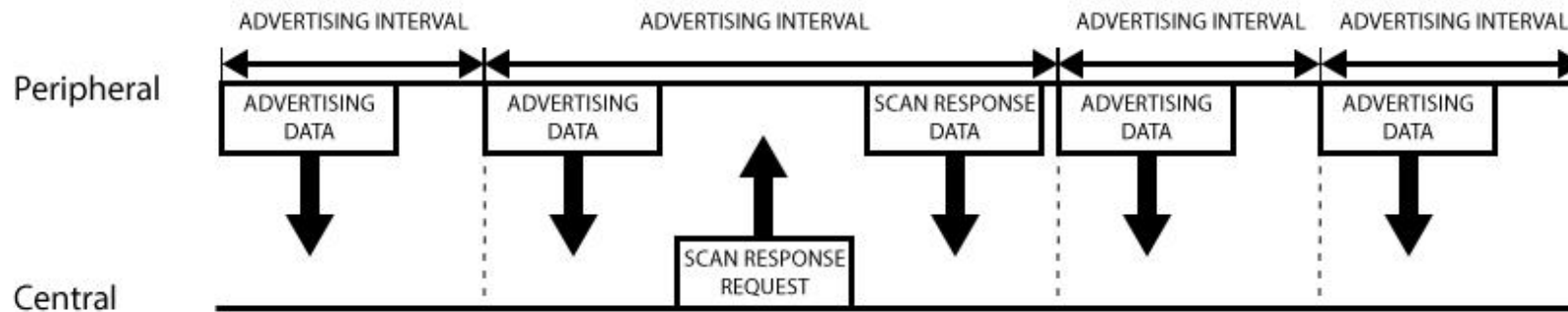
So, see the difference?

- Advertisement-based vs. Connection-based:

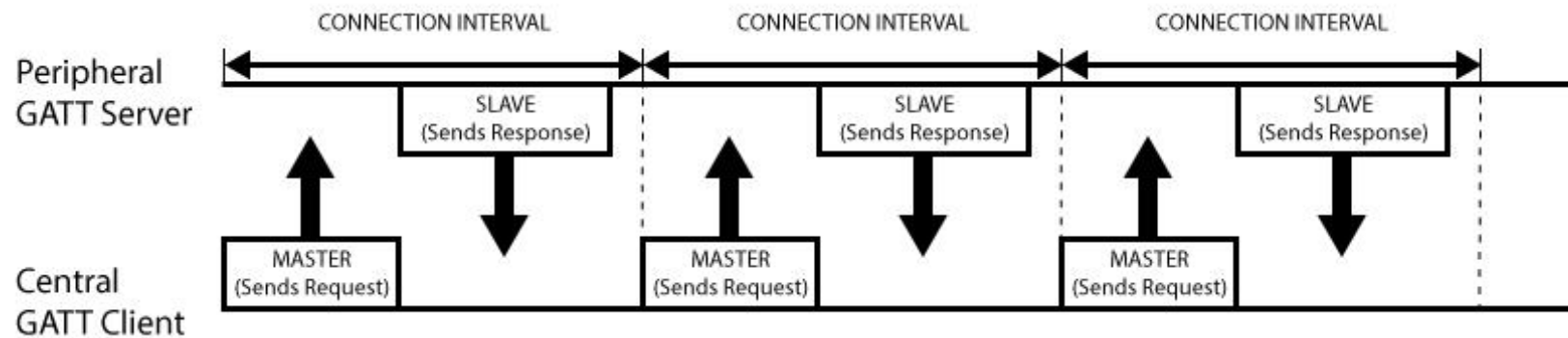


Again, see the difference?

- Advert



Connection-less, broadcast communication



Connection-oriented, two-way communication



References

- https://en.wikipedia.org/wiki/Bluetooth_low_energy
- <https://learn.adafruit.com/introduction-to-bluetooth-low-energy/introduction>
- <https://developer.android.com/guide/topics/connectivity/bluetooth-le.html>
- <http://www.rfwireless-world.com/Terminology/Bluetooth-vs-BLE.html>
- <https://www.link-labs.com/blog/bluetooth-vs-bluetooth-low-energy>





Linux

Hcitol commands



- hcitol is used to configure Bluetooth connections and send some special command to Bluetooth devices.

Command	Function
hcitol -i hciX	Use the interface hciX for the command. If not specified, defaults to the first available interface
hcitol scan	Scans for classic Bluetooth devices which are in discoverable mode
hcitol lescan	Scans for BLE devices

More commands: <https://helpmanual.io/man1/hciconfig/>

Gatttool commands



Command	Function
<code>gatttool -I</code>	Launch gatttool in interactive mode
<code>gatttool -t random -b [adr] -I</code>	Launch gatttool in interactive mode using a random LE address. Connect to the remote Bluetooth device having address adr.
<code>primary</code>	Check for available services of the connected BLE device
<code>characteristic</code>	Check for available characteristics of the connected BLE device from where we can read data
<code>char-desc</code>	Characteristics Descriptor Discovery. Check for n no of handles
<code>char-read-hnd</code>	Read characteristic by their handle
<code>char-write-req</code>	Write values to the handle

More commands: <https://helpmanual.io/man1/gatttool/>

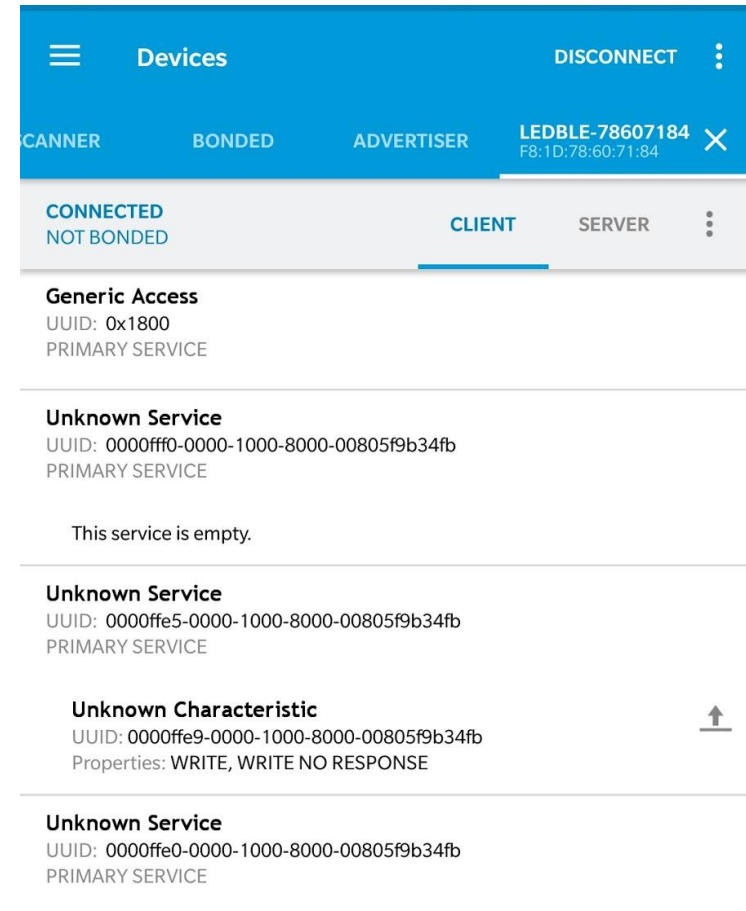
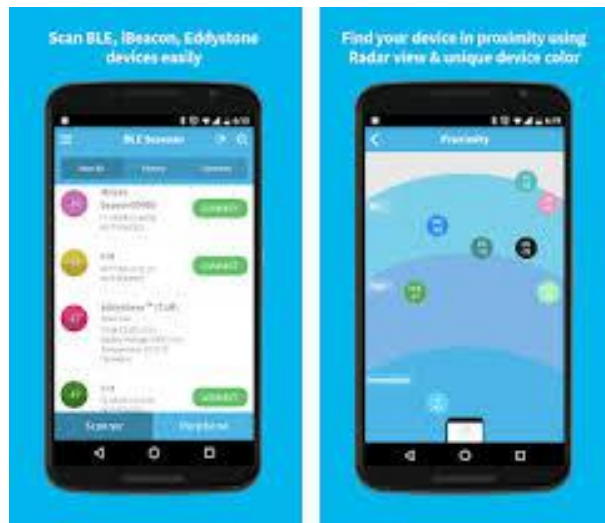
- GATT stands for Generic Attribute and
- defines a data structure for organizing characteristics and attributes.
- We can discover, read, and write characteristics using gatttool.

- `hciconfig` : Used to list all the attached BLE adapters.
- `hciconfig hciX up` : Enable the BLE adapter named hciX.
- `hciconfig hciX down` : Disable the BLE adapter named hciX.
- `hcitool lescan` : Scan for BLE devices in the vicinity.
- `gatttool -I` : Launches gatttool in an interactive REPL like mode where the user can various issue commands as listed below.
 - `connect <addr>` : Connect to the BLE device with the specified address.
 - `gatttool -t random -b <addr> -I` : Connect to the device using a random address.



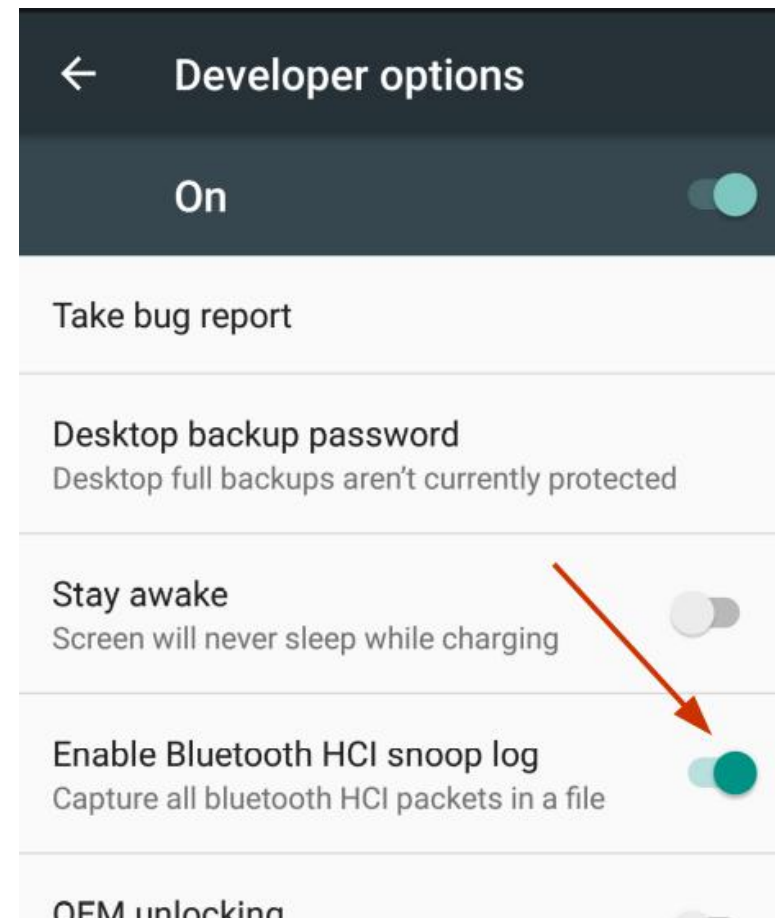
Apps uteis

- Apps
- Exemplos:
 - nRF Connect
 - BLE Scanner



Bluetooth hci snoop log

- Android starting from version 4.4 onwards has an option to record all bluetooth packets going in/out from the device. To enable capture of bluetooth traffic follow the steps below. Ensure that the android app is installed.
- Step 1 : Go to mobile Settings and enable the developer option.
- Step 2 : Go to "developer option" and enable Bluetooth HCI snoop log
- Step 3 : Run the android app (magic blue) app and send some commands to the bulb to change its color. Repeat this several times.
- Step 4 : Corresponding to our commands we can find the captured file of the Bluetooth traffic at /sdcard/btsnoop_hci.log or /internal Storage/btsnoop_hci.log
- Note - In some devices, btsnoop_hci.log is created in a different location like /sdcard/Android/data/btsnoop_hci.log.
- Transfer the captured log file to a computer/laptop using Email, Google Drive or you can connect the android device to your laptop via USB cable.
- Step 4 : Analyze the capture packets in Wireshark. Wireshark is a free and open source packet analyzer tool and can be installed by running
- <https://blog.attify.com/exploiting-iot-enabled-ble-smart-bulb-security/>



Mais Info

- <https://blog.attify.com/the-practical-guide-to-hacking-bluetooth-low-energy/>
- <https://mcuoneclipse.com/2016/12/19/tutorial-ble-pairing-the-raspberry-pi-3-model-b-with-hexiwear/>
- <https://www.jaredwolff.com/get-started-with-bluetooth-low-energy/#!>
- <https://www.jaredwolff.com/get-started-with-bluetooth-low-energy/#!>



Fim

- Dúvidas?
- Comentários?

